

COMP2522

Lesson 7: Method References, Generics, Nested and Inner Classes

Learning Outcomes

- Method references
- Generics
- Bounded type parameters ("bounded generics")
- Upper-bounded generics (extends)
- Lower-bounded generics (super)
- Wildcards in generics (method parameters and collections)
- 4 types of nested classes

Method references

- Method references are a **shorthand syntax which replaces lambda expression**
- Method references use the `::` operator without any parameters
- They can be used to make simple lambda expressions by making use of existing methods.

Four Types of Method References

- There are four types of method references:
 - A method reference to a static method
 - `ClassName::staticMethod`
 - A method reference to an instance method of an object of a certain type
 - `ClassName::instanceMethod`
 - A method reference to an instance method of an existing object
 - `object::instanceMethod`
 - A method reference to a constructor
 - `ClassName::new`
- See next slide for examples of each

Examples

```
class MathUtils {
    static int add(int a, int b) { return a + b; }
}

class Printer {
    void printMessage() { System.out.println("Hello!"); }
}

class Car {
    Car() { System.out.println("Car created!"); }
}

public class Main {
    public static void main(final String[] args) {
        // Static Method Reference
        BiFunction<Integer, Integer, Integer> adder = MathUtils::add;
        System.out.println(adder.apply(3, 4)); // 7

        // Instance Method Reference (on a specific object)
        Printer printer = new Printer();
        Runnable<Void> message = printer::printMessage;
        message.get(); // Hello!

        // Instance Method Reference (on any object of a type)
        BiFunction<String, String, Integer> comparator = String::compareTo;
        System.out.println(comparator.apply("apple", "banana")); // Negative value (-1)

        // Constructor Reference
        Supplier<Car> carSupplier = Car::new;
        carSupplier.get(); // Car created!
    }
}
```

Lambda vs. method reference

- `(args)->Class.staticMethod(args)`
- ...can be replaced by...
- `Class::staticMethod`
- Any arguments will be passed automatically by Java

Lambda vs. method reference: 0 args

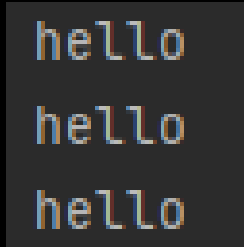
```
import java.util.concurrent.Callable;
import java.util.function.Consumer;
class Notebook{
    public static void main(String[] args) {
        sayHello();

        // lambda expression with zero input arguments
        final Runnable stringConsumer3 = () -> sayHello(); // Runnable: // zero arguments; no return
        stringConsumer3.run();

        // method reference
        final Runnable stringConsumer4 = Notebook::sayHello;
        stringConsumer4.run();
    }

    public static void printNote(final String note) {
        System.out.println(note);
    }

    public static void sayHello() {
        System.out.println("hello");
    }
}
```



hello
hello
hello

Lambda vs. method reference: 1 arg

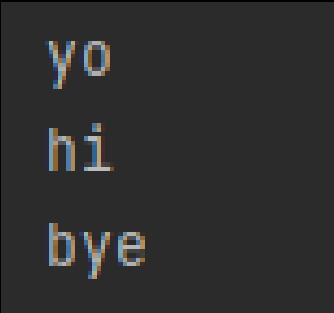
```
import java.util.function.Consumer;

class Notebook
{
    public static void main(String[] args)
    {
        printNote("yo");

        // lambda expression with one input argument
        final Consumer<String> stringConsumer1 = (String s) -> printNote(s);
        stringConsumer1.accept("hi");

        // method reference
        final Consumer<String> stringConsumer2 = Notebook::printNote;
        stringConsumer2.accept("bye");
    }

    public static void printNote(final String note)
    {
        System.out.println(note);
    }
}
```



yo
hi
bye

```

import java.util.Arrays;
import java.util.List;
import java.util.function.Consumer;
import java.util.function.Predicate;
class People{
    public static void main(final String[] args) {
        List<Person> people = Arrays.asList(
            new Person("tiger", "woods", 1975),
            new Person("oprah", "winfrey", 1961)
        );

```

Inner class with lambda

```

        doConditionalStuff(people, p->true, p-> System.out.println(p));
    }
    private static void doConditionalStuff(List<Person> people, Predicate<Person> predicate, Consumer<Person> consumer) {
        for(Person pers: people) {
            if(predicate.test(pers)) {
                consumer.accept(pers);
            }
        }
    }
    private static class Person {
        private final String firstName;
        private final String lastName;
        private final int yearBorn;
        public Person(final String firstName, final String lastName, final int yearBorn) {
            this.firstName = firstName;
            this.lastName = lastName;
            this.yearBorn = yearBorn;
        }
        @Override
        public String toString() {
            return firstName + " " + lastName + " was born in " + yearBorn;
        }
    }
}

```

```

tiger woods was born in 1975
oprah winfrey was born in 1961

```

```

class People{

    public static void main(final String[] args) {
        List<Person> people = Arrays.asList(
            new Person("tiger", "woods", 1975),
            new Person("oprah", "winfrey", 1961)
        );
    }

```

```

    doConditionalStuff(people, p->true, System.out::println);
}

```

```

private static void doConditionalStuff(List<Person> people, Predicate<Person> predicate, Consumer<Person> consumer) {
    for(Person pers: people) {
        if(predicate.test(pers))
        {
            consumer.accept(pers);
        }
    }
}

```

```

private static class Person {
    private final String firstName;
    private final String lastName;
    private final int yearBorn;

    public Person(final String firstName, final String lastName, final int yearBorn) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.yearBorn = yearBorn;
    }
}

```

```

@Override
public String toString() {
    return firstName + " " + lastName + " was born in " + yearBorn;
}
}

```

Inner class with method reference

```

tiger woods was born in 1975
oprah winfrey was born in 1961

```

NOTE: The method reference `System.out::println` uses the `System.out` instance to get at the NON-STATIC method `println`. This is taking the place of the `Consumer` in the `doConditionalStuff` method, which takes a parameter of a `Person` in this case.

```
class More{
    public static void main(final String[] args) {
        final List<Strings> titles;
        titles = new ArrayList();
        titles.add("power of now");
        titles.add("can't hurt me");
        titles.add("discipline equals freedom");
        titles.add("meditations");
        titles.forEach(t-> System.out.println(((String)t).toUpperCase()));

        titles.forEach(System.out::println);
    }
}
```

Generics

- Generics allow us to write methods, classes, interfaces, and collections that enforce type safety at compile-time.

Without Generics: Using Object

```
public class BoxWithoutGenerics {
    private Object t;          // stores any reference type
    public void set(final Object t) { this.t = t; }
    public Object get() { return t; }
}

public class Main {
    public static void main(final String[] args) {
        final BoxWithoutGenerics box = new BoxWithoutGenerics();

        box.set("Hello"); // Storing a String
        final String s = (String)box.get(); // Explicit casting is needed

        box.set(123); // Storing an Integer
        final Integer i = (Integer) box.get(); // Explicit casting again
    }
}
```

- **We can store any object type! BUT:**
- Problems without Generics:
- Explicit Casting: You need to cast the result of get() to the expected type, which increases the risk of runtime ClassCastException errors if you mistakenly cast to the wrong type.
- Lack of Type Safety: There's no compile-time checking to ensure the type you're setting matches the type you're getting. For example, you might mistakenly store an Integer and attempt to retrieve it as a String, which would cause a runtime error.

With Generics: Using Box(T)

```
// T's type defined at compile time and erased at runtime
public class Box<T> {
    private T t;
    public void set(final T t) { this.t = t; }
    public T get() { return t; }
}

public class Main {
    public static void main(final String[] args) {
        final Box<String> stringBox = new Box<>();
        stringBox.set("Hello");    // Store a String
        String s = stringBox.get(); // No casting needed

        Box<Integer> integerBox = new Box<>();
        integerBox.set(123);        // Store an Integer
        final Integer i = integerBox.get(); // No casting needed
    }
}
```

Let the compiler enforce type safety whenever possible.

- Three advantages of using generics:
 - 1. No Casting Required
 - Box<String> knows that it must store and return String objects
 - Box<Integer> knows that it must handle Integer objects.
 - 2. Type Safety (reduced runtime errors)
 - Generics provide compile-time type checking.
 - If you try to store a String in a Box<Integer>, the compiler will throw an error, preventing that mistake:
- Box<Integer> integerBox = new Box<>();
- integerBox.set("Hello"); // **Compile-time error:** incompatible types
- 3. Reusability
- Use the same Box<T> class for any type (String, Integer, CustomObject, etc.), reducing code duplication and making classes more versatile.

Generics: More Type Parameters

- `class Pair<T, U> {`
- `private final T first;`
- `private final U second;`
- `public Pair(final T first, final U second) {`
- `this.first = first;`
- `this.second = second;`
- `}`
- `public T getFirst() { return first; }`
- `public U getSecond() { return second; }`
- `}`
- T and U are "type parameters", and can represent any reference type
- T and U are often used for types
- E for elements
- K and V for key and Value
- BUT
- We can use any names we like:
- `class Pair<FirstType, SecondType>{`
- `private final FirstType first;`
- `...`
- `class Pair<Key, Value>`
- `class Pair<Instructor, Course>`

Generics: More Type Parameters

```
class Pair<T, U> {  
    private final T first;  
    private final U second;  
  
    public Pair(final T first,  
                final U second) {  
        this.first = first;  
        this.second = second;  
    }  
  
    public T getFirst() { return first; }  
    public U getSecond() { return second; }  
}
```

```
final Pair<String, Integer> pair1 = new Pair<>("one", 1);  
final Pair<String, Integer> pair2 = new Pair<>("twenty six", 26);  
  
final Pair<String, String> pair3 = new Pair<>("dodge", "charger");  
final Pair<String, String> pair4 = new Pair<>("volkswagen", "beetle");  
  
final Pair<Company, Person> pair5 = new Pair(new  
    Company("facebook", 2004), new Person("zuckerberg", 1984));  
final Pair<Company, Person> pair6 = new Pair(new  
    Company("microsoft", 1975), new Person("gates", 1955));  
  
final Pair<Double, String> pair7 = new Pair(95.0, "A+");  
final Pair<Double, String> pair8 = new Pair(50.0, "Pass");
```

Generic Methods

- Generic methods introduce their own type parameters, independent of the class type parameters. This allows for flexibility when performing actions that don't depend on the class's generic type.
- Here, the generic method works with any type that implements Comparable.

```
public class Utility {  
    public static <T extends Comparable<T>> T getMax(T a, T b)  
    {  
        return a.compareTo(b) > 0 ? a : b;  
    }  
}
```

(more on "extends" soon)

Bounded Types in Generics

- You can use bounds to restrict generic types with the extends or super keywords. This ensures that only certain specific types can be passed while still allowing flexibility.
- Example:
 - `public <T extends Number> void addNumbers(T a, T b) {`
 - `System.out.println(a.doubleValue() + b.doubleValue());`
 - `}`
- In this example, the method accepts only types that extend Number, ensuring type safety with numeric types.

Bounded Types with Classes

```
class Pair<T, U> {  
    private final T first;  
    private final U second;  
  
    public Pair(final T first,  
                final U second) {  
        this.first = first;  
        this.second = second;  
    }  
  
    public T getFirst() { return first; }  
    public U getSecond() { return second; }  
}
```

- This generic typing allows T and U to be of absolutely *any reference type* at all
- **Instead, we can put upper- or lower-bounds on the allowed types**
- UPPER BOUND: restricts a type parameter to a specific type or its subclasses using the extends keyword
- LOWER BOUND: restricts a wildcard to a specific type of its superclasses using the super keyword

Upper Bound

- An upper bound in Java generics restricts the type of a generic parameter to a specific class **or its subclasses**
- It is defined using the extends keyword
- This allows type safety, ensuring that only certain types are used
- This guarantees access to specific methods; we can use methods available in the upper-bound class

```
class Box<T extends Number> { // T must be Number or a subclass of Number
    private T value;

    public T getValue(){
        return value; // returns the original value in its actual type
    }
    public void setValue(final T value) {
        this.value = value;
    }
    public double getDoubleValue() {
        return value.doubleValue(); // Can call methods from Number
    }
}

public class Main {
    public static void main(final String[] args) {
        final Box<Integer> intBox;
        final Box<Double> doubleBox;
        final Box<Float> floatBox;    // any Number subclass is ok
        // final Box<String> strBox;    // Compilation error: String does not extend Number
        // final Box<Object> strBox;    // Compilation error: Object does not extend Number

        intBox = new Box<>();
        doubleBox = new Box<>();

        intBox.setValue(10);
        System.out.println("Double value: " + intBox.getDoubleValue());

        doubleBox.setValue(5.5);
        System.out.println("Double value: " + doubleBox.getDoubleValue());
    }
}
```

Bounded Types with methods

```
public <T extends Number> void addNumbers(T a, T b) {  
    System.out.println(a.doubleValue() + b.doubleValue()); // doubleValue() is a Number method  
}  
  
addNumbers(5, 10);           // ok  
addNumbers(5.0, 10.0);       // ok  
addNumbers(5.0f, 10.0f);     // ok  
addNumbers("5", "10");       // no; String is not a Number  
addNumbers(true, false);     // no; Boolean is not a Number
```

- In this example, the method accepts only types that extend `Number`, ensuring type safety with numeric types.

Wildcards in Generics

- Wildcards provide flexibility in generics, allowing a method to accept a range of types. There are three common types of wildcards:
 - ? Unbounded wildcard
 - ? extends T Upper-bounded wildcard (T or any subclass)
 - ? super T Lower-bounded wildcard (T or any superclass)
- Example:
 - ```
public void printList(final List<?> list) {
```
  - ```
    for(final Object elem : list) {
```
 - ```
 System.out.println(elem);
```
  - ```
    }
```
 - ```
}
```
  - See next slides for use cases

# Unbounded Wildcard ?

- `printList(List<?> list)` can accept a List of any type, but the actual type of the elements is unknown. The wildcard helps make the method more flexible, allowing it to work with lists of different types without compromising type safety.
- `List<?>` means that the method can accept a List of any specific type (e.g., `List<String>`, `List<Integer>`, `List<MyCustomType>`, etc.). However, inside the method, you don't **know or care** about the actual type of elements in the list.
- Type Flexibility: You can pass any list (e.g., `List<String>`, `List<Integer>`, etc.) to this method.
- Type Safety (at compile time): While the method can work with any list, it restricts what you can do with the list to prevent unsafe operations. For instance, **you cannot add elements to the list** because the method doesn't know the actual type of elements in the list.
- Using `List<?>` makes sense when you want to write a method that can operate on a list of any type but doesn't need to modify the list. For example: just printing the elements without needing to know or alter their types.

- ```
public void printList(List<?> list) {  
    for(final Object elem : list) {  
        System.out.println(elem);  
    }  
    /* the next line is a compilation error; cannot add  
       elements because the type is unknown, BUT we  
       can *read* elements from the list because every  
       type in Java is ultimately a subtype of Object. */  
    // list.add("Hello");  
}
```
- ```
public static void main(final String[] args) {
 List<String> stringList = List.of("A", "B", "C");
 List<Integer> intList = List.of(1, 2, 3);

 printList(stringList); // Works with a list of Strings
 printList(intList); // Works with a list of Integers
}
```
- Note that the last two lines would NOT have compiled if we had defined `printList`'s argument as `List<Object>` (see next slide)

# Unbounded Wildcard vs. Object

- `List<Object>` means the method **only accepts a list of Objects**, which excludes `List<String>`, `List<Integer>`, and other types.
  - Why? Because `List<String>` is not a subtype of `List<Object>`. Generics in Java are invariant, meaning `List<String>` and `List<Object>` are considered completely different types.
  - This would cause a compile-time error if you try to pass a `List<String>` to a method expecting `List<Object>`.
  - `List<?>`, on the other hand, means the method can accept a list of any type (e.g., `List<String>`, `List<Integer>`, etc.).
- ```
public void printObjectList(List<Object> list) {  
    for (Object elem : list) {  
        System.out.println(elem);  
    }  
}
```
 - ```
List<String> stringList = List.of("A", "B", "C");
printObjectList(stringList); // Compilation error!
// List<String> is not List<Object>
```

# Multiple Bounds in Generics

- Multiple bounds allow you to specify that a type must satisfy multiple conditions.
- Example:
  - `public <T extends Comparable<T> & Serializable> T getMax(T a, T b) {`
  - `return a.compareTo(b) > 0 ? a : b;`
  - `}`
  - Here, T must be both Comparable and Serializable.
- The syntax for multiple bounds in generics is:
  - `<T extends Bound1 & Bound2 & Bound3>`
  - The first bound (Bound1) must always be a **class or** an interface.
  - After the first bound, any additional bounds must be interfaces.
  - You can have any number of interfaces, but you can only extend one class (since Java doesn't allow multiple inheritance of classes).

# Generics and Inheritance

- Generics and inheritance interact in specific ways
- A `List<Dog>` is not a subtype of `List<Animal>`, even though `Dog` is a subtype of `Animal`.
- Example:
  - `List<Dog> dogs = new ArrayList<>();`
  - `// List<Animal> animals = dogs; // Compile-time error`

# Covariance and Contravariance

- Covariance (? **extends** T) allows for accepting subtypes of T
- Contravariance (? **super** T) allows for accepting supertypes of T

Covariance: We can pass lists of Integer, Double, Float, ... since the upper bound is Number

```
public void printNumbers(List<? extends Number> numbers) {
 for (Number n : numbers) {
 System.out.println(n);
 }
}
```

- Covariance allows us to pass a collection (or array) of objects that are subtypes of a certain type T.
- i.e. We can read objects of type T (or its subtypes) from the collection, but cannot add objects to it because doing so could violate type safety.
- Covariance is useful when you want to consume items from a collection without modifying it
- It is a compilation error to add to the list; e.g. may be adding a Double to a List<Integer>).

# Contravariance

- Covariance (? extends T) allowed for accepting subtypes of T
- Contravariance (? super T) allows for **supertypes**.
- Contravariance: We can pass lists of Integer, Number, or Object, ... since the lower bound is Integer
- ```
public void addNumbers(List<? super Integer> list) {  
    list.add(10);  
    list.add(20);  
    // error; not a subtype of Integer:  
    // list.add(1.5);} 
```
- Contravariance allows us to pass a collection (or array) of objects that are supertypes of a certain type T.
- i.e. The collection can accept objects of type T or any of its subtypes.
- We can safely add elements to a contravariant collection, but you can't necessarily read specific types from it because the compiler cannot guarantee what the list holds at runtime; we must assume it's "Object" since that's the common super class.

PECS Principle

- To remember when to use covariance or contravariance, use the PECS rule:

- **Producer Extends, Consumer Super**

- Producer
 - Reading (e.g. from a list)
 - ? extends
 - upper-bounded wildcard
 - The collection/array is producing data
 - Will return data of type T or subtype of T
 - Cannot write since the exact type is not known
- Consumer:
 - Writing (e.g. to a list)
 - ? super
 - lower-bounded wildcard
 - The collection is being added to
 - Cannot read from it since the only returned types may only be of type Object

- Remember the getMax() method we wrote?

```
static <T extends Comparable<T>> T getMax(final T[] a) {
    T max = a[0];
    for(final T i: a) {
        if(i.compareTo(max) > 0) {
            max = i;
        }
    }
    return max;
}
```

```
/* static int getMax(final int[] a) {
    int max = a[0];
    for(int i: a) {
        if(i > max) {
            max = i;
        }
    }
    return max;
}*/
}
```

```
public class Max{
    public static void main(final String[] args) {
        final Integer[] ints = {5, 44, 321, -6, 0, 31, 34, 300};
        final Character[] chars = {'a', 'g', 'z', 'w', 't'};
        final Double[] doubles = {5.5, 44.44, 321.321, -6.6, 0.0, 31.31, 34.34, 300.3};
        final Dog[] dogs = {new Dog(20, 2000), new Dog(14, 2014), new Dog(17, 2017)};
        final String[] strings = {"hello world", "the end", "how are you", "whatever", "bye"};

        System.out.println(getMax(ints));
        System.out.println(getMax(chars));
        System.out.println(getMax(doubles));
        System.out.println(getMax(dogs));
        System.out.println(getMax(strings));
    }
}
```

PECS

```
class Animal {}  
class Dog extends Animal {}
```

```
public class PECSExample {  
    public static void main(final String[] args) {  
        List<Dog> dogs = new ArrayList<>();  
        dogs.add(new Dog());  
  
        // Producer: ? extends Animal (reading from a list)  
        List<? extends Animal> animals = dogs;  
        Animal animal = animals.get(0); // Can read an Animal  
  
        // Cannot write to a producer (compiler error)  
        // animals.add(new Dog()); // Error: can't add to a ? extends collection  
  
        // Consumer: ? super Dog (writing to a list)  
        List<? super Dog> animalList = new ArrayList<>();  
        animalList.add(new Dog()); // Can add a Dog  
  
        // Cannot safely read from a consumer (compile error)  
        // Dog dog = animalList.get(0); // Error: can't guarantee it's a Dog  
    }  
}
```

- - Producer (`? extends`):
- - You can **read** from the list (`animals.get(0)`), and the returned value is guaranteed to be of type `Animal` or its subtype.
- - You **cannot add** (`animals.add(new Dog())`) because the exact type in the list is not known.
-
- - Consumer (`? super`):
- - You can **add** (`animalList.add(new Dog())`) because it's safe to add `Dog` or its subtype.
- - You **cannot safely read** from the list (`animalList.get(0)`) because the return type could be any supertype of `Dog`, and the compiler only guarantees `Object`.

PECS

```
public class PECSExample {
    // Producer (reading) - ? extends T (covariant)
    public static void read(List<? extends Number> producer) {
        Number n = producer.get(0); // Allowed (can read as Number)
        // producer.add(10);          // Compilation error (cannot write)
    }

    // Consumer (writing) - ? super T (contravariant)
    public static void write(List<? super Integer> consumer) {
        consumer.add(10);           // Allowed (can insert Integer)
        // Integer i = consumer.get(0); // Compilation error (cannot read)
    }

    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3);
        read(numbers); // Producer (reading)

        List<Number> numList = new ArrayList<>();
        write(numList); // Consumer (writing)
    }
}
```

- Why This Works:
- **? extends T (Producer)**
 - Can read values (as T or a superclass like Number).
 - Cannot write values (except null).
- **? super T (Consumer)**
 - Can write values (T and its subclasses).
 - Cannot read specific types (only Object).

Generics do not follow standard inheritance rules

- In Java, generics are invariant, meaning `List<Integer>` is NOT a subtype of `List<Number>`, even though `Integer` extends `Number`.
- Why? If `List<Integer>` were treated as `List<Number>`, we could do this:
- `List<Number> numList = new ArrayList<Integer>();` // Not allowed!
- `numList.add(3.14);` // Would break type safety (Double in Integer list)
- This would corrupt the `List<Integer>`.
- To prevent this, Java enforces strict type safety in generics.
- How to Work Around It?
- Use `?` extends `Number` for reading (but can't add elements)
- Use `?` super `Integer` for writing (but retrieved elements are `Object`)

Nested Classes

- We can define classes inside other classes.
- These are called nested classes.
- If a nested class is not static, it is called an inner class.
- Nested classes can have access modifiers such as private, protected, or public.
- A nested class can also be static, meaning it does not require an instance of the outer class.
- If a class is only relevant to one other class, it is often a good candidate for being an inner (nested) class, which keeps functionality encapsulated.
- Separating functionality into well-defined classes, whether nested or not, helps improve code readability and maintainability.

Nested Classes

- Java's nested classes are used to logically group classes that are only used in one place, improving encapsulation and organization by keeping related code together while accessing the outer class's members, including private ones.
- There are four types of inner classes:
 1. Static nested classes
 2. Non-static inner classes
 3. Local inner classes (inside methods)
 4. Anonymous inner classes

Static / Non-Static

```
class Car {
    private final String model = "Tesla"; // Outer class field

    static class CarInfo { // Static nested class (no access to instance members)
        public void displayInfo() {
            System.out.println("This is a static nested class example.");
        }
    }

    class Engine { // Non-static inner class (can access instance members)
        public void start() {
            // Accessing the outer class's instance field 'model'
            System.out.println("Engine started in " + model); // Accesses outer class member
        }
    }
}

public class Main {
    public static void main(final String[] args) {
        Car.CarInfo info = new Car.CarInfo(); // no need to create a Car
        info.displayInfo();

        Car car = new Car();
        Car.Engine engine = car.new Engine(); // requires a Car
        engine.start(); // Now accesses the outer class's 'model' field
    }
}
```

Local Inner Classes

- Local inner classes are **defined inside methods** and have access to local variables only if they are marked as final or effectively final (i.e., Java ensures the value is never changed; this ensures that the inner class does not access a variable that might later be modified, which could cause unexpected behavior).
- Java enforces the use of final or effectively final variables because these classes or constructs can outlive the method in which they were defined. If a local variable's value changed after the inner class was created, it could cause inconsistencies since the inner class might still refer to the old value. By requiring variables to be final or effectively final, Java ensures that the inner class can safely capture the variable's value, knowing it won't change.
- This example simulates a common banking system operation, where transactions are processed and logged internally without exposing unnecessary details. The local inner class is used to keep the logging logic scoped within the method.

```
class BankAccount {  
    private String accountHolder;  
    private double balanceCad;  
    public BankAccount(String accountHolder, double balanceCad) {  
        this.accountHolder = accountHolder;  
        this.balanceCad = balanceCad;  
    }  
}
```

```
• }
```

```
public void processTransaction(final double amountCad) {  
    // Local variable marked as effectively final (no modification)  
    final String transactionType = (amountCad > 0) ? "Deposit" : "Withdrawal";  
    // Local inner class inside method  
    class TransactionLogger {  
        public void logTransaction() {  
            System.out.println("Account Holder: " + accountHolder);  
            System.out.println("Transaction Type: " + transactionType);  
            System.out.println("Transaction Amount Cad: " + amountCad);  
            System.out.println("New Balance Cad: " + balanceCad);  
        }  
    }  
    balanceCad += amount;  
    // Create an instance of the local inner class and log the transaction  
    final TransactionLogger logger = new TransactionLogger();  
    logger.logTransaction();  
}
```

```
public class Main {  
    public static void main(final String[] args) {  
        final BankAccount account = new BankAccount("John Doe", 1000.00);  
        account.processTransaction(500); // Deposit  
        account.processTransaction(-200); // Withdrawal  
    }  
}
```

Anonymous Inner Class

- An **anonymous inner class** in Java is a type of inner class without a name, often used to instantiate classes or interfaces on the fly, usually when you need to override a method or provide a specific implementation for an interface or abstract class. They are commonly used in event handling, threading, or small use-case scenarios where a full class declaration would be overkill.
- **EXAMPLE:** sort a list of integers in descending order using an anonymous inner class to define a custom comparator.

```
public class Main {  
    public static void main(final String[] args) {  
        List<Integer> numbers = new ArrayList<>();  
        numbers.add(10);  
        numbers.add(5);  
        numbers.add(20);  
        numbers.add(15);  
  
        // Sort list in desc order with a nonymous inner class for Comparator  
        Collections.sort(numbers, new Comparator<Integer>() {  
            @Override  
            public int compare(Integer a, Integer b) {  
                return Integer.compare(b, a); // Sort in descending order  
            }  
        });  
  
        // Print the sorted list  
        System.out.println(numbers); // Output: [20, 15, 10, 5]  
    }  
}
```

Comparator

- The Comparator interface lets us define multiple ways to compare objects; it also lets us use a comparison logic that is external to the class being compared.
- It allows us to create separate classes or methods with different comparison strategies, so we can define different ways of ordering the same class without modifying its source code.
- A Comparator Can be passed to sorting methods like Collections.sort() or Arrays.sort().

```
class Person {
    private final String name;
    private final int yearBorn;
    public Person(final String name, final int yearBorn) {
        this.name = name;
        this.yearBorn = yearBorn;
    }
    @Override
    public String toString() {
        return name + " (" + yearBorn + ")";
    }
}
```

```
public static void main(final String[] args) {
    final List<Person> people = new ArrayList<>();
    people.add(new Person("Alice", 2000));
    people.add(new Person("Bob", 1999));
    people.add(new Person("Charlie", 1988));
```

```
    Collections.sort(people, new Comparator<Person>() {
        @Override
        public int compare(Person p1, Person p2) {
            return p1.yearBorn - p2.yearBorn;
        }
    });
```

```
    System.out.println("Sorted by age: " + people);
```

```
    // Sort by name using another Comparator
    Collections.sort(people, new Comparator<Person>() {
        @Override
        public int compare(Person p1, Person p2) {
            return p1.name.compareTo(p2.name);
        }
    });

    System.out.println("Sorted by name: " + people);
}
```

Law of Demeter

- The **Law of Demeter (LoD)** is also known as the "**Principle of Least Knowledge**"
- Reduces the dependencies between classes and ensure that an object only interacts with its close "friends" (other objects it directly interacts with), not with objects from distant layers.
- **"Only talk to your immediate friends."**
- This means an object should only call methods on:
 1. **Itself**
 2. **Its own fields** (its member variables)
 3. **Objects passed in as arguments** to its methods
 4. **Objects it creates locally** inside its methods
- An object **should not**:
 - Reach into another object's internals and call methods on the internal objects of those objects (this is considered "talking to strangers").
 - Chain method calls across multiple objects (e.g., `a.getB().getC().doSomething()`).
 - This reduces coupling between classes, making the code more maintainable, easier to change, and less prone to errors.

Breaking the LoD

- class Engine {
- public void start() {
- System.out.println("Engine started");
- }
- }
-
- class Car {
- private Engine engine = new Engine();
-
- public Engine getEngine() {
- return engine;
- }
- }
-
- class Driver {
- public void startCar(Car car) {
- // Violates the Law of Demeter by calling a method on the engine via car.
- **car.getEngine().start();** // BAD: Driver is directly accessing the engine of the car.
- }
- }
- In the above example:
- The Driver class directly interacts with the Engine by first calling car.getEngine() and then engine.start().
- This breaks the Law of Demeter because the driver is "reaching through" the car to interact with the engine, which it shouldn't need to know about.

Following the LoD

- `class Engine {`
- `public void start() {`
- `System.out.println("Engine started");`
- `}`
- `}`
-
- `class Car {`
- `private Engine engine = new Engine();`
-
- `// Car provides a method to start the engine, hiding internal details`
- `public void startEngine() {`
- `engine.start(); // Car knows about its engine and controls it`
- `}`
- `}`
-
- `class Driver {`
- `public void startCar(Car car) {`
- `// Now Driver is only interacting with Car, not with the Engine directly.`
- `car.startEngine(); // GOOD: Car handles the engine starting internally.`
- `}`
- `}`
- In this correct example:
- The Driver class only interacts with the Car by calling `car.startEngine()`, which is allowed under the Law of Demeter.
- The Car manages the interaction with its Engine internally, so the Driver doesn't need to know about the Engine.

LoD Benefits

- **Key Benefits:**
 - **Reduced Coupling:** By following LoD, you minimize the dependency between classes, making your code more modular and flexible.
 - **Easier Maintenance:** Changes in one class are less likely to affect other classes because there are fewer interdependencies.
 - **Improved Readability:** The code is easier to understand since objects are only interacting with their immediate "friends."